Weather-Based Prediction of Rising Waters:Floods

---

1. Introduction

- **Project Title: Rising Waters: A Machine Learning Approach To Flood Prediction**
- **Team Members:**
  - **Syed Mohammed Ali**
  - **Shaik Masthan Valli**
  - **Katika Mahammad Meharaan**
  - **Boya Nikhil Naidu**

---

2. Project Overview

- **Purpose:**
  The purpose of this project is to build a machine learning–based system that predicts Rising Waters Flood
- **Features:**
  - **Data preprocessing and cleaning.**
  - **Random Forest regression model for prediction.**
  - **Flask-based web dashboard for user interaction.**
  - **Integration with OpenWeather API for real-time weather data.**
  - **Visualization of actual vs predicted power outputs.**

---

3. Architecture

- **Frontend:**
  Flask templates (HTML, CSS, JavaScript) used for UI design and dashboard visualization.
- **Backend:**
  Python Flask application handling API requests, ML model predictions, and weather data integration.
- **Database:**
  Local CSV dataset (T1.csv) for training and testing. Model stored as .sav file using Joblib. Future scope includes cloud database integration (MongoDB Atlas / AWS RDS).

---

4. Setup Instructions

- **Prerequisites:**
  - **Python 3.9+**
  - **Flask**
  - **Pandas, NumPy, Scikit-learn, Matplotlib**
  - **Joblib**
  - **OpenWeather API key**
- **Installation:**

1. **Clone the repository.**
2. **Install dependencies using pip install -r requirements.txt.**
3. **Set up environment variables (API key for OpenWeather).**
4. **Run the Flask server with python app.py.**

---

5. Folder Structure

- **Client (Frontend):**
  - **templates/ → HTML files (intro page, dashboard).**
  - **static/ → CSS, JS, and images.**
- **Server (Backend):**
  - **app.py → Flask application.**
  - **model_training.py → ML model training script.**
  - **power_prediction.sav → Saved Random Forest model.**
  - **data/ → Dataset files.**

6. Running the Application

- **Frontend: Runs automatically via Flask templates.**
- **Backend: Start with python app.py.**

---

7. API Documentation

- **Endpoint 1: /predict**
  - **Method: POST**
  -
  - **Response: Predicted Active Power**
- **Endpoint 2: /weather**
  - **Method: GET**
  - **Parameters: City name**
  - **Response: Weather data (temperature, humidity, pressure, wind speed)**

---

8. Authentication

- **Currently open access.**
- **API key required for OpenWeather API integration.**
- **Future scope: JWT-based authentication for secure access.**

---

9. User Interface

- **Intro page with project overview.**
- **Dashboard with:**
  - **Rain ata display.**
  - **Prediction module.**
  - **Visualization graphs.**

10. Testing

- **Unit testing for ML model predictions.**
- **API testing for weather data retrieval.**
- **Functional testing for input validation and dashboard navigation.**

11. Screenshots or Demo

- **Scatter plot of actual vs predicted Floods.**

- **Dashboard screenshot showing weather + prediction results.**
- **Line chart trends of Rain Fall vs Floods Output.**

---

12. Known Issues

- **Limited dataset size may affect generalization.**
- **API errors if invalid city names are entered.**
- **No authentication for dashboard access (future enhancement needed).**

---

13. Future Enhancements

- **Deploy on cloud (AWS/GCP/Azure).**
- **Add JWT authentication for secure access.**
- **Expand dataset for improved accuracy.**
- **Add more visualizations (heatmaps, time-series forecasting).**
- **Integrate grid demand APIs for real-time energy balancing.**

---

14. Conclusion

The *Rising Waters* project demonstrates that **machine learning can significantly enhance flood prediction accuracy and timeliness**, addressing the limitations of conventional forecasting systems. By analyzing historical weather data, river levels, terrain features, and population density, the system predicts flood probabilities and provides **early warnings** to authorities and communities, enabling proactive disaster management. The use of **Random Forest classifiers** and techniques like **SMOTE** to handle data imbalance ensures robust model performance, with evaluation metrics such as accuracy, precision, recall, and F1-score confirming its reliability.